

Clean Code Development — Personal Cheat Sheet

Requirement 6B — >10 points, own words, applied while working on Sentinel-Core / AGENTIC_PROJECT

1. Meaningful Names

Names should reveal intent without needing a comment. If I have to explain a variable name out loud, it's the wrong name.

2. Functions Do One Thing

A function should be describable without using 'and'. ``analyze_vitals`` checking two conditions is borderline — worth watching as it grows.

3. Avoid Magic Values

Raw literals like ``"TACHYCARDIA_EVENT"`` scattered in code are a trap — one typo silently breaks matching logic. Named constants centralize the truth.

4. Fail Loud, Not Silent

Swallowing an error into a ``printf`` and returning ``void`` hides failures from callers. Return a status code or propagate the error.

5. Avoid Implicit Conversions

Compiler warnings like ``-Wconversion`` exist for a reason — an implicit ``time_t`->unsigned int`` narrowing should be an explicit, visible cast.

6. Single Level of Abstraction per Function

Mixing low-level string formatting with high-level business logic in one function makes both harder to read in isolation.

7. DRY (Don't Repeat Yourself)

Duplicated threshold-check patterns are a sign the logic wants to be a small table or loop, not N copy-pasted ``if`` blocks.

8. Comments Explain Why, Not What

A comment restating what the next line obviously does is noise. A comment explaining a non-obvious constraint (e.g. platform quirk) earns its place.

9. Small, Focused Commits

A commit that mixes an unrelated formatting pass with an actual bugfix makes ``git blame`` useless later.

10. Tests as Documentation

A well-named test (``test_normal_vitals_no_alert``) tells a reader what the function is supposed to NOT do — something the implementation alone can't show.

11. Boy Scout Rule

Leave the code a little cleaner than you found it — e.g. wiring up a dead function (``export_to_json``) instead of leaving it orphaned.

12. Prefer Composition over Hard-coded Coupling

Passing the alert mechanism as a function pointer/callback instead of hard-coding ``printf`` inside the analysis function keeps concerns separated and testable.

13. Immutability Where Possible

``const`` fields on ``Vitals``, or ``Object.freeze`` in JS, remove a whole class of accidental-mutation bugs at compile/runtime rather than by convention.

14. Explicit Over Implicit Configuration

A hard-coded relative file path breaks the moment the working directory changes; reading it from an environment variable with a sane fallback makes the assumption visible and overridable.